

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA0104**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A
Algorithm Design Challenge

Code of Conduct:

I _K.Nagapreetham(192525242)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	

Total	100%	
-------	------	--

Signature of the Faculty
the Questions

Total Marks Awarded **Answer all**

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given Romania Map Problem graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning

5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents
---	---	--

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.

9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

Q1. Algorithm Design – Greedy Best-First Search

1. Problem Statement:

Find a path from the source city to the destination using Greedy Best-First Search.

2. Problem Analysis:

- Initial State: Source city
- Goal State: Destination city
- Constraint: Avoid revisiting explored nodes.

3. AI Technique Selection:

Greedy Best-First Search (GBFS) – an informed search using heuristic values.

4. Justification:

GBFS selects the node that appears closest to the goal. It is fast but does not guarantee the shortest path.

5. State Space Representation:

Cities are nodes and roads are edges. The node with the lowest heuristic is expanded.

6. Variables, Domains and Constraints:

Not Applicable – not a CSP problem.

7. Heuristic Function:

$$f(n) = h(n)$$

where $h(n)$ is the estimated distance to the goal.

8. Algorithm / Pseudocode:

1. Put Start in OPEN.
2. Select node with minimum $h(n)$.
3. If node = Goal, return path.
4. Expand the node.
5. Add unvisited successors to OPEN.
6. Repeat until Goal is reached.

9. Flowchart / Algorithm Diagram

Insert GBFS flowchart here.

10. Working / Execution Trace

Start → **Calculate ($h(n)$)** → **Select lowest ($h(n)$)** → **Expand** → **Repeat** → **Goal**.

11. Complexity Analysis:

Time: $(O(b^m))$

Space: $(O(b^m))$

12. Advantages and Limitations

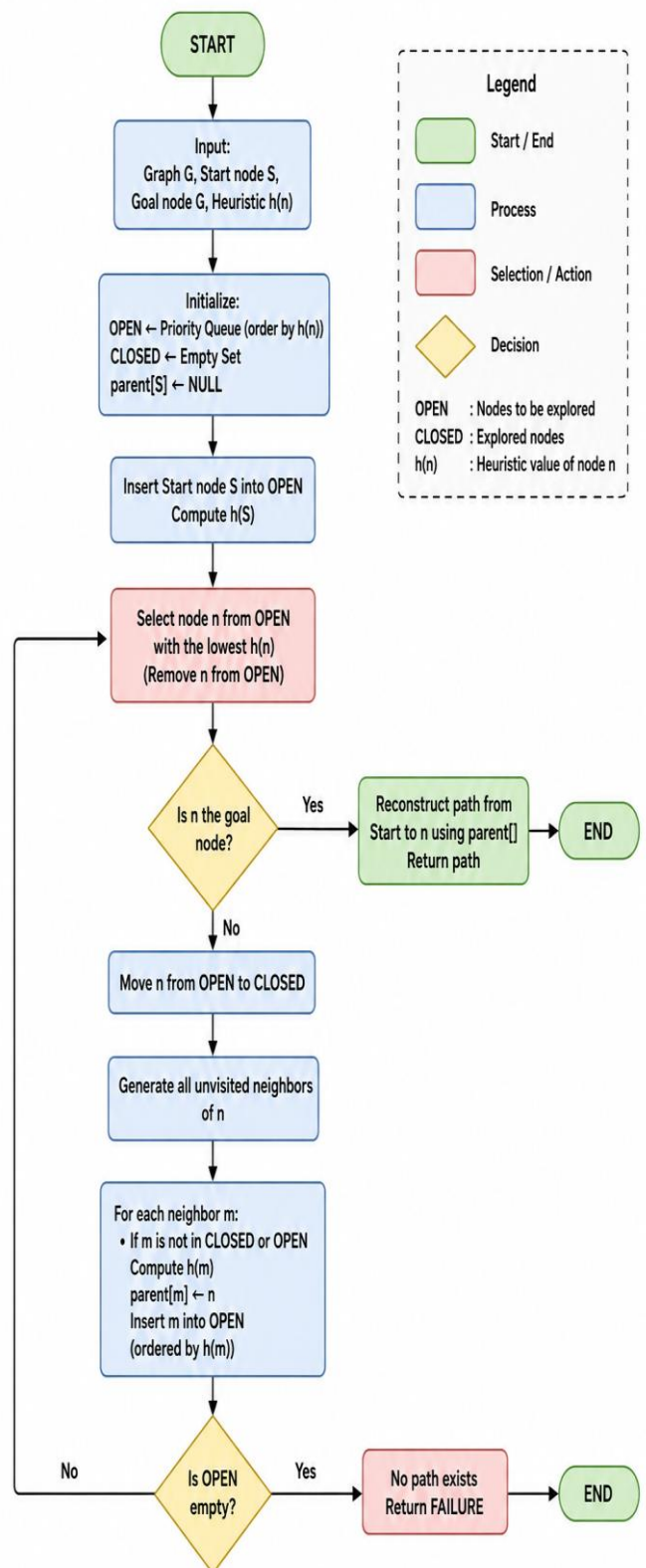
Advantages: Simple and fast.

Limitations: Not always optimal.

13. Conclusion:

GBFS provides a goal-directed search using heuristic information and is suitable when fast search is preferred.

Greedy Best-First Search Algorithm Flowchart



Q2 – A* SEARCH:

1. Problem Statement:

Find the optimal path for a delivery robot from the warehouse to the customer.

2. Problem Analysis:

- Initial State: Warehouse
- Goal State: Customer
- Constraint: Minimum travel cost.

3. AI Technique Selection:

A* Search – combines actual and estimated costs.

4. Justification:

A* is suitable because it can find an optimal path when the heuristic is admissible.

5. State Space Representation:

Locations are nodes and weighted roads are edges.

6. Variables, Domains and Constraints:

Not Applicable – weighted path-search problem.

7. Heuristic Function:

$$f(n) = g(n) + h(n)$$

($g(n)$) is the cost travelled and ($h(n)$) is the estimated remaining cost.

8. Algorithm / Pseudocode:

1. Put Start in OPEN.
2. Calculate $f(n) = g(n) + h(n)$.
3. Select node with minimum $f(n)$.
4. If node = Goal, return path.
5. Expand neighbors and update their costs.
6. Repeat until Goal is reached.

9. Flowchart / Algorithm Diagram:

10. Working / Execution Trace:

Calculate ($g(n)$), ($h(n)$) and ($f(n)$) → Select minimum ($f(n)$) → Expand → Repeat → Goal.

11. Complexity Analysis:

Time: ($O(b^d)$)

Space: ($O(b^d)$)

12. Advantages and Limitations:

Advantages: Can find an optimal path.

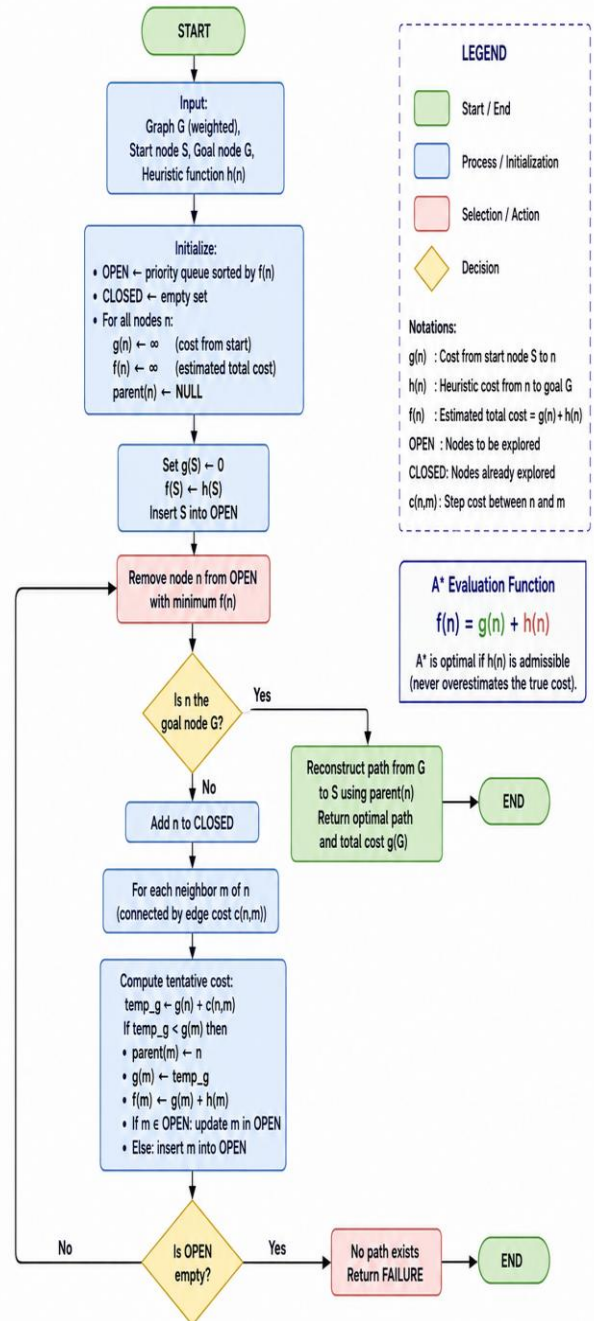
Limitations: Requires more memory.

13. Conclusion:

A* efficiently finds an optimal route by considering both travelled and estimated costs.

A* SEARCH ALGORITHM FLOWCHART

(Optimal Path in Weighted Road Network)



Q3.CSP WITH MRV AND FORWARD CHECKING

1. Problem Statement:

Create a university examination timetable without scheduling conflicting subjects together.

2. Problem Analysis:

- Variables: Subjects
- Domains: Examination slots
- Goal: Valid timetable
- Constraint: Conflicting subjects need different slots.

3. AI Technique Selection:

CSP with MRV and Forward Checking.

4. Justification:

CSP models subjects, slots and conflicts directly. MRV selects the most constrained subject first.

5. State Space Representation:

Subjects are variables, slots are values, and conflicts are constraints.

6. Variables, Domains and Constraints:

Variables: Subjects

Domains: (S1, S2, S3, ...)dots)

Constraint: Conflicting subjects cannot share a slot.

7. Heuristic Function:

MRV selects the variable having the fewest remaining legal values.

8. Algorithm / Pseudocode:

1. Select subject using MRV.
2. Assign a valid slot.
3. Apply Forward Checking.
4. Remove the slot from conflicting subjects.
5. If a domain becomes empty, backtrack.
6. Repeat until all subjects are assigned.

10. Working / Execution Trace:

Select most constrained subject → Assign slot → Remove conflicting slots → Check → Backtrack if needed → Continue.

11. Complexity Analysis:

Worst-case: $O(d^n)$

12. Advantages and Limitations:

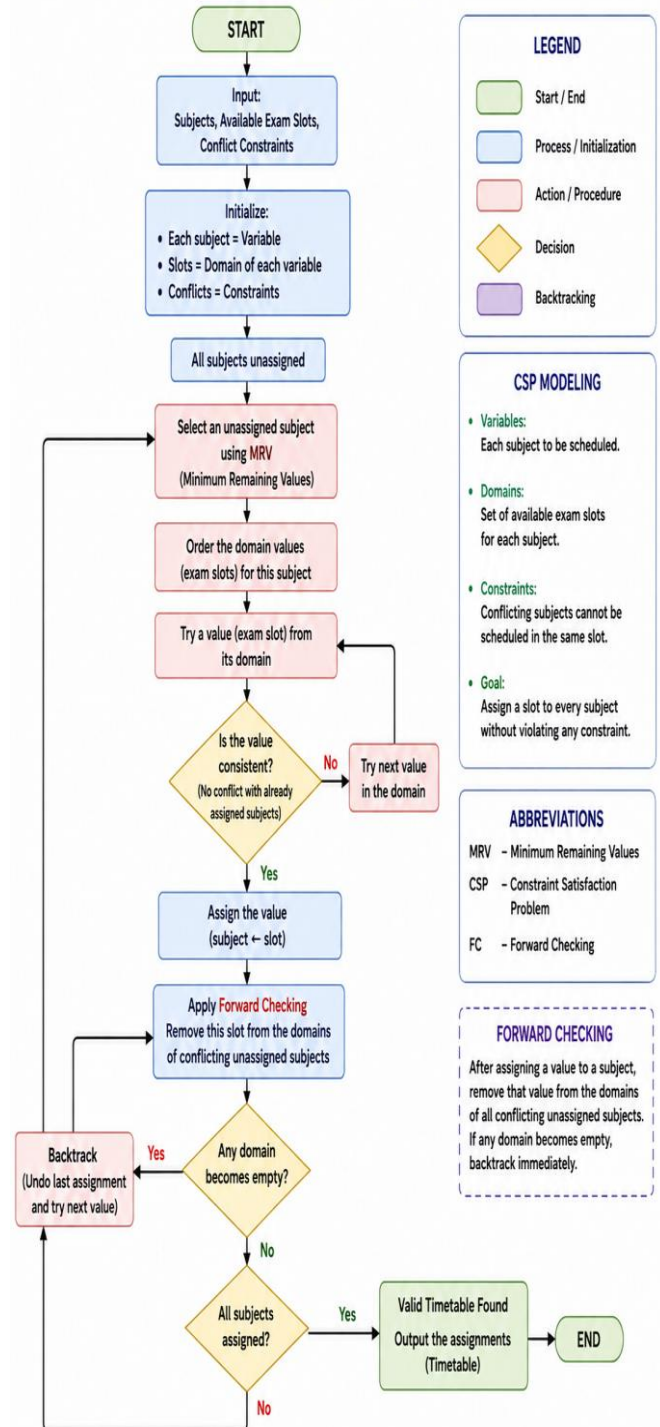
Advantages: Reduces conflicts and unnecessary search.
Limitations: Can be expensive for large timetables.

13. Conclusion:

CSP with MRV and Forward Checking efficiently generates a valid examination timetable.

CSP - Examination Timetable (Using MRV & Forward Checking)

Flowchart / Algorithm Diagram



Q4 – MINIMAX WITH ALPHA-BETA PRUNING

1. Problem Statement:

Find the best move for the MAX player using a game tree.

2. Problem Analysis:

- MAX tries to maximize the score.
- MIN tries to minimize the score.
- Both players are assumed to play optimally.

3. AI Technique Selection:

Minimax with Alpha-Beta Pruning.

4. Justification:

Minimax finds the best move, while Alpha-Beta removes branches that cannot affect the final decision.

5. State Space Representation:

The game is represented as a tree containing MAX nodes, MIN nodes and terminal values.

6. Variables, Domains and Constraints:

Not Applicable – game-tree search problem.

7. Heuristic Function:

Not Applicable when terminal nodes already contain utility values.

8. Algorithm / Pseudocode:

1. If terminal node, return its value.
2. MAX selects maximum child value.
3. MIN selects minimum child value.
4. Update α and β .
5. If $\alpha \geq \beta$, prune the remaining branch.
6. Return the best value.

10. Working / Execution Trace:

Evaluate leaves $\rightarrow$ MAX selects maximum $\rightarrow$ MIN selects minimum $\rightarrow$ Update α , $\beta \rightarrow$ Prune unnecessary branches $\rightarrow$ Select best move.

11. Complexity Analysis:

Minimax: $O(b^d)$

Alpha-Beta best case: $O(b^{\lceil d/2 \rceil})$

12. Advantages and Limitations:

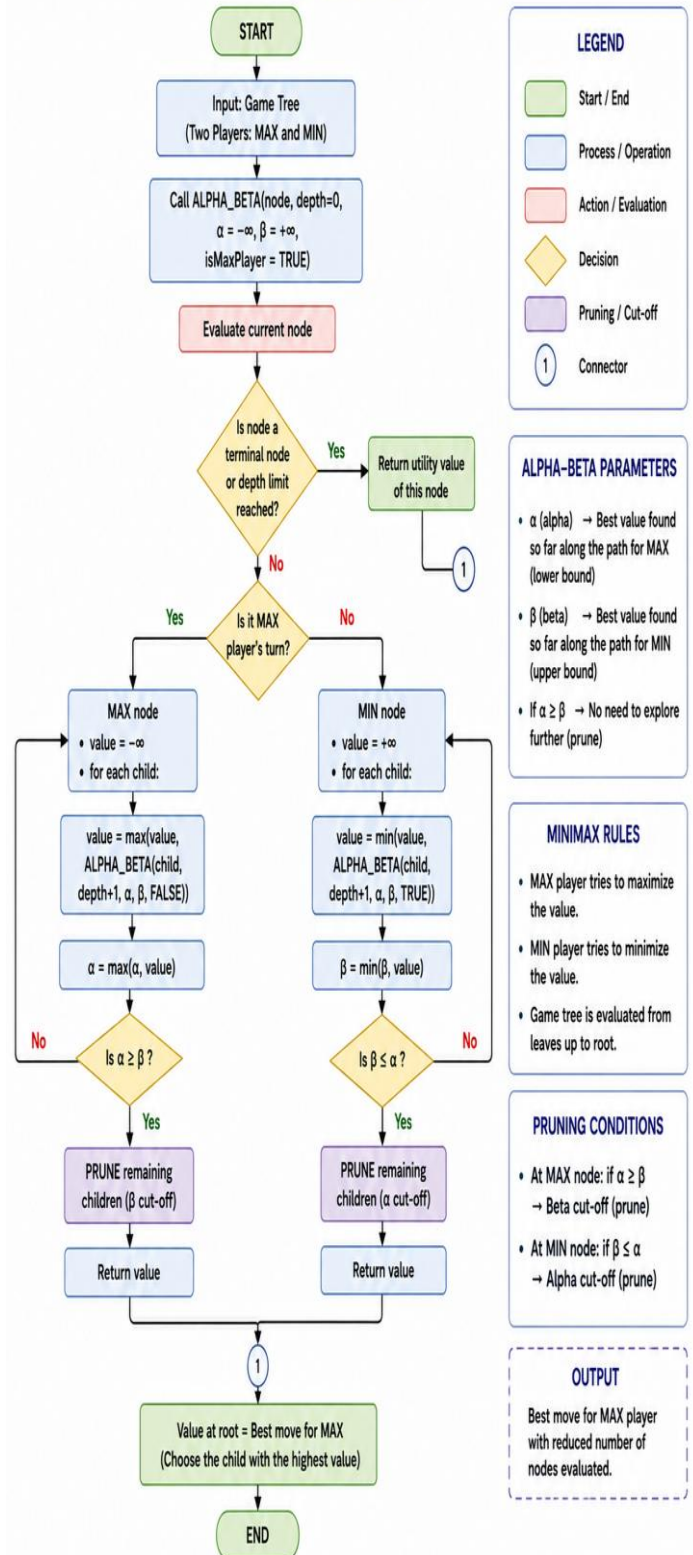
Advantages: Reduces node evaluation and improves speed.

Limitations: Performance depends on move ordering.

13. Conclusion:

Minimax finds the optimal move, while Alpha-Beta Pruning improves efficiency without changing the result.

MINIMAX ALGORITHM WITH ALPHA-BETA PRUNING FLOWCHART



Q5 – ONLINE SEARCH AGENT

1. Problem Statement:

Design a search strategy for a warehouse robot that must handle unexpected obstacles.

2. Problem Analysis:

- Initial State: Robot's current position
- Goal State: Destination
- Constraint: Avoid obstacles and adapt to changes.

3. AI Technique Selection:

Online Search Agent.

4. Justification:

An online agent senses the environment while moving and can replan when obstacles appear.

5. State Space Representation:

Warehouse locations are states and possible movements are transitions. Obstacles change available paths.

6. Variables, Domains and Constraints:

Not Applicable – not a CSP problem.

7. Heuristic Function:

For a grid:

$$h(n) = |x_n - x_g| + |y_n - y_g|$$

This estimates the distance to the goal.

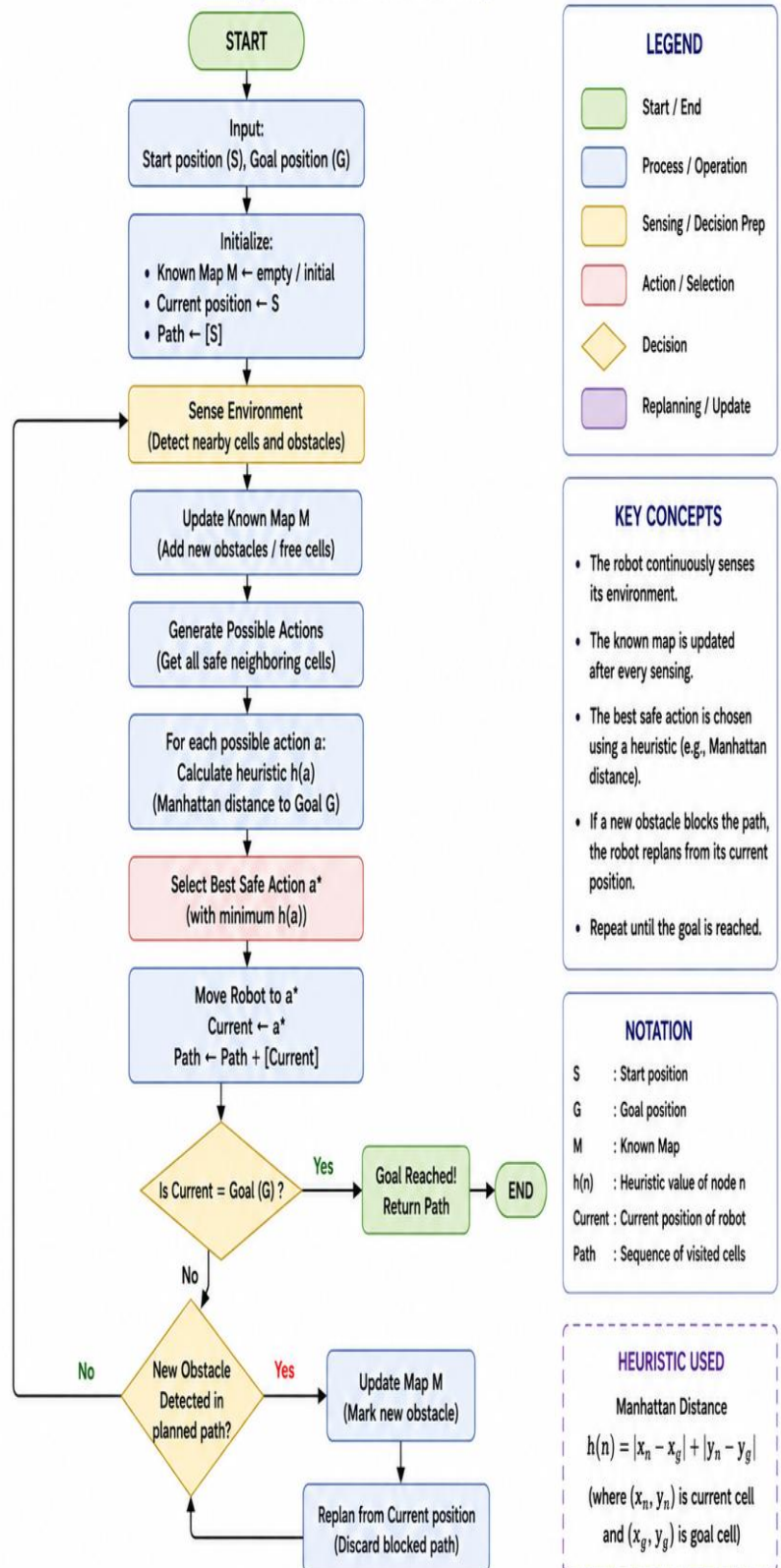
8. Algorithm / Pseudocode:

1. Start at the initial position.
2. Sense the environment.
3. Update the map.
4. Select the best safe next state.
5. Move the robot.
6. If an obstacle appears, replan.
7. Repeat until the goal is reached.

10. Working / Execution Trace:

Sense → Update map → Select safe path → Move → Detect obstacle → Replan → Continue → Goal.

ONLINE SEARCH AGENT FLOWCHART (Dynamic Warehouse Robot)



11. Complexity Analysis: Worst-case search can be $O(b^d)$, depending on the environment and replanning. 12. Advantages and Limitations: Advantages: Adaptive and suitable for dynamic environments. Limitations: Replanning requires additional computation. 13. Conclusion: Online Search is suitable for warehouse robots because it continuously senses, updates and replans its path.	
---	--

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.